DECLARATIVE CANVAS LAYOUT ENGINE

Sone

Build rich PDFs & images declaratively with a fluent Flexbox API

Node.js | Browser | Multi-Page PDF | Flexbox Layout

FEATURES

Column / Row / Grid
Flexbox layout containers

Text & Span
Rich inline typography

TextDefault
Inherit text styles

Photo
Images with scaleType

Path
SVG paths with fills

Table / TableRow / TableCell
Structured data tables

List / ListItem
Ordered & unordered lists

ClipGroup
SVG clip paths

Multi-Page PDF
pageHeight, header, footer

Transforms
rotate, scale, translate

Filters
blur, grayscale, invert...

QR Codes
qrcode() built-in

Sone Documentation · April 2026

01 — QUICK START

Installation & Basic Usage

Everything you need to render your first layout.

Install

```
npm install sone
# or
pnpm add sone
```

Minimal example

```
import { Column, Row, Text, sone } from 'sone';

const root = Column(
  Text("Hello, World!").size(32).weight("bold"),
  Row(
    Text("Left").flex(1),
    Text("Right"),
  ).gap(16).marginTop(12),
).padding(40).bg("white");

await sone(root).png();
// ⟶ Buffer (PNG)
```

sone() — output formats

Method	Returns
.png()	Promise<Buffer> — PNG image
.jpg(quality?)	Promise<Buffer> — JPEG (0.0 – 1.0)
.pdf()	Promise<Buffer> — PDF (multi-page when pageHeight set)
.svg()	Promise<Buffer> — SVG vector
.webp()	Promise<Buffer> — WebP image
.raw()	Promise<Buffer> — Raw pixel data
.canvas()	Promise<SoneCanvas> — recorded canvas instance

SoneRenderConfig

Option	Description

width / height	Override canvas dimensions
background	Canvas background color
pageHeight	Enable multi-page output (px per page)
header / footer	Static node or (SonePageInfo) => SoneNode
margin	number or { top, right, bottom, left }
lastPageHeight	"uniform" | "content"
cache	Map for image caching across renders

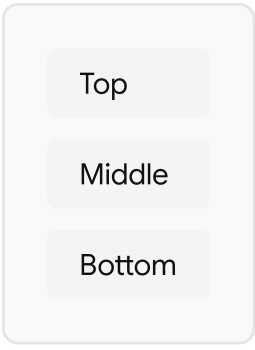
02 — LAYOUT

Column, Row & Flexbox

All layout containers are flexbox-based. Column stacks children vertically; Row lays them out horizontally.

Column — vertical stack

```
Column(
  Text("Top"),
  Text("Middle"),
  Text("Bottom"),
).gap(8).padding(16).bg("white")
```



Children stacked top-to-bottom.

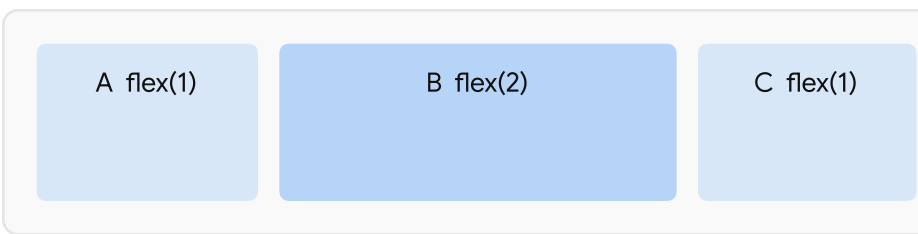
.gap(n) sets spacing between items.

.alignItems() controls cross-axis.

.justifyContent() controls main-axis.

Row — horizontal stack

```
Row(
  Text("A").flex(1),
  Text("B").flex(2),
  Text("C").flex(1),
).gap(8)
```



.flex(n) distributes remaining space.

.shrink(0) prevents compression.

.wrap() enables line wrapping.

.alignSelf() overrides per-child.

Layout properties reference

flex(n)	Flex shorthand	flexGrow / grow(n)	Grow factor
flexShrink / shrink(n)	Shrink factor	flexBasis / basis(n)	Base size
gap / rowGap / columnGap	Spacing between children	padding(...)	CSS shorthand (1–4 values)
margin(...)	CSS shorthand (1–4 values)	width / height	Fixed or 'auto' or '%'
size(w, h?)	Set both at once (square if h omitted)	minWidth / maxWidth	Size constraints
position()	static | relative | absolute	inset / top / left...	Absolute offsets
alignItems / alignSelf	Cross-axis alignment	justifyContent	Main-axis alignment
alignContent	Multi-line alignment	wrap()	Enable flex wrapping
overflow('hidden')	Clip overflowing children	display('none')	Hide without removing from layout

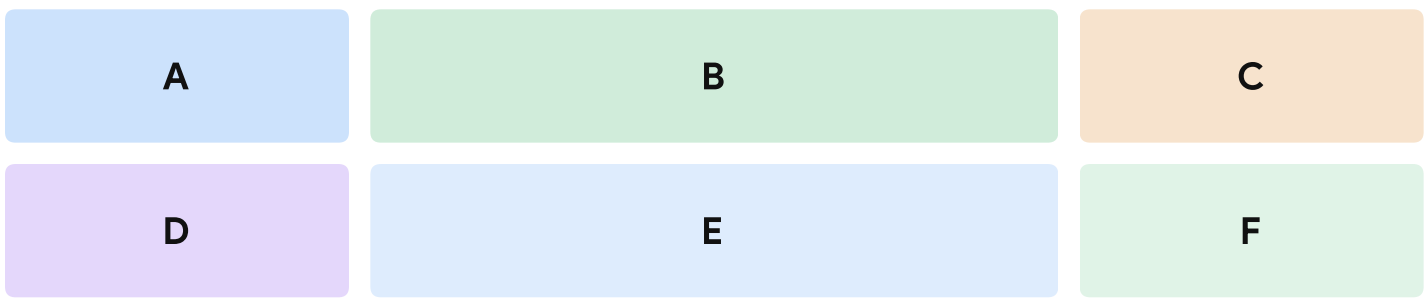
03 — GRID

CSS Grid Layout

Grid supports fixed, auto, and fractional (fr) track sizes, explicit placement, and spanning.

Basic grid

```
Grid(
  Column(Text("A")),  // auto-placed
  Column(Text("B")),
  Column(Text("C")),
)
  .columns("1fr", "2fr", "1fr")
  .columnGap(12)
  .rowGap(12)
```



Tracks: fixed(px), auto, fr

.columns(...) — column sizes

.rows(...) — row sizes

.autoRows() — implicit rows

.columnGap() / .rowGap()

Explicit placement & spanning

```
Grid(
  Column(Text("Hero")).gridColumn(1, 2).gridRow(1),  // spans 2 cols
  Column(Text("Side")).gridColumn(3).gridRow(1),
  Column(Text("Full width")).gridColumn(1, 3),     // spans all 3
).columns("1fr", "1fr", "200px")
```



.gridColumn(start, span?)

.gridRow(start, span?)

Explicit placement mixes

Full Width
gridColumn(1, 3) · spans all

with auto-flow.

Track size types

150
Fixed pixels

"auto"
Fit content

"2fr"
Fractional

04 — TEXT

Text, Span & TextDefault

Rich inline text with per-span styling, decorations, gradients, and advanced layout options.

Text — basic

```
Text("Hello world")       // plain string
Text("Hello", " world")   // multiple strings/spans
Text(Span("bold").weight("bold"), " normal")
```

Hello world

Center

Right

Justify — this line is long enough to show justification in action here

.size(px) — font size

.color(value) — text color

.weight() — bold, 100–900

.font(...) — font stack

.style('italic') — font style

.align() — left/center/right/justify

.lineHeight(n) — line height multiplier

.lineBreak() — greedy | knuth-plass

Span — inline styling

```
Text(
  Span("Underline").underline(1.5),
  Span("Strike").lineThrough(),
  Span("Overline").overline(),
  Span("Highlight").highlight("yellow"),
  Span("Shadow").dropShadow("2px 2px 4px rgba(0,0,0,0.3)"),
  Span("Stroke").strokeColor("blue").strokeWidth(1.5),
  Span("Raised").offsetY(-6),
)
```

Underline ~~Strike~~ Overline

Highlight Shadow Stroke

Raised normal Lowered Wide

Text layout options

nowrap()
Prevent line wrapping

maxLines(n)
Clamp visible lines

textOverflow('ellipsis')
Show … when clamped

indent(px)
First-line indent

hangingIndent(px)
Subsequent lines indent

tabStops(...px)
Tab stop positions

tabLeader(char)
Fill gaps e.g. "." for dot leader

orientation(0|90|180|270)
Layout-aware rotation

autofit()
Scale font to fit container

clipImage(Photo(...))
Image through letterforms

wordSpacing(px)
Extra space between words

letterSpacing(px)
Extra space between chars

TextDefault — inherit text styles

```
TextDefault(
  Column(
    Text("Inherits size & color"),
    Text("from TextDefault"),
  ),
).size(14).color("navy")
```

Inherits size & color

From TextDefault

Override per node

05 — STYLING

Backgrounds, Shadows, Filters & Transforms

Any layout node can have rich visual styling applied via the fluent builder API.

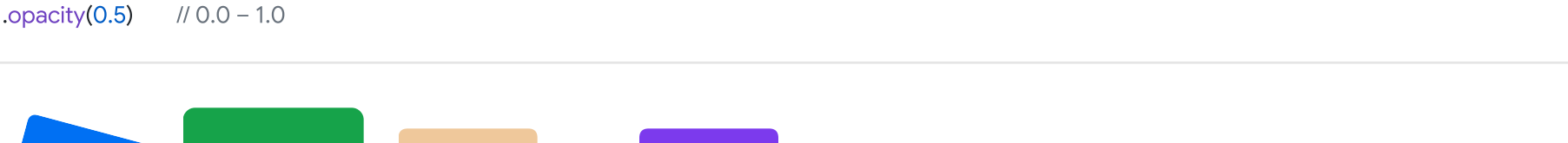
Background — colors, gradients, images

```
Column().bg("red")
Column().bg("linear-gradient(135deg, #667eea, #764ba2)")
Column().bg("radial-gradient(circle, yellow, orange)")
Column().bg(Photo("image.jpg").scaleType("cover"))  // image bg
```



Shadows

```
Column().shadow("0 4px 12px rgba(0,0,0,0.15)")
Column().shadow("2px 2px 0px black")       // hard
Column().shadow("0 0 20px #0070f3")        // glow
```



CSS Filters

```
.blur(4)        // blur(4px)
.brightness(1.4)  // brightness(1.4)
.contrast(1.5)    // contrast(1.5)
.grayscale(1)     // grayscale(1)
.invert(1)        // invert(1)
.sepia(1)         // sepia(1)
.saturate(2)      // saturate(2)
.huerotate(180)   // hue-rotate(180)
```


Transforms

```
.rotate(45)       // rotation in degrees
.scale(1.2)       // uniform scale
.scale(1.5, 0.8)  // x / y scale
.translateX(20)   // horizontal offset
.translateY(-10)  // vertical offset
.opacity(0.5)     // 0.0 – 1.0
```





06 — BORDERS & CORNERS

Per-Side Borders and Corner Styles

borderWidth accepts 1–4 values (CSS shorthand). Corners support radius, squircle smoothing, and cut style.

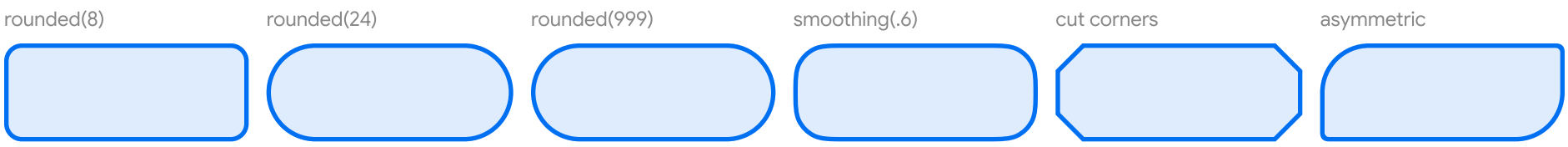
borderWidth — CSS shorthand

```
.borderWidth(2)          // all 4 sides
.borderWidth(2, 8)       // top/bottom=2, left/right=8
.borderWidth(2, 8, 4)    // top=2, left/right=8, bottom=4
.borderWidth(2, 4, 6, 8) // top, right, bottom, left
```



Corner styles

```
.rounded(8)              // uniform radius
.rounded(8, 0, 8, 0)     // top-left, top-right, bottom-right, bottom-left
.cornerSmoothing(0.6)    // squircle (0 – 1)
.corner("cut")           // chamfered cut corners
```



overflow('hidden') — clip children

```
Row(
  Photo("image.jpg").size(200, 100),
).rounded(16).overflow("hidden")  // clips to rounded border
```



overflow('hidden') clips child content to the node's border-box. Works with cornerRadius as well.

07 — PHOTO, PATH & CLIPGROUP

Images, SVG Paths and Clip Groups

Render images with flexible scaling, draw SVG paths with gradient fills, and clip arbitrary shapes.

Photo — image display

```
Photo("./image.jpg").size(200, 120).scaleType("cover")
Photo(buffer).scaleType("contain").preserveAspectRatio()
Photo("icon.svg").size(32).fill("gray")  // fallback color
```



Photo props:

· src: url | Buffer

· scaleType: cover | contain | fill

· preserveAspectRatio

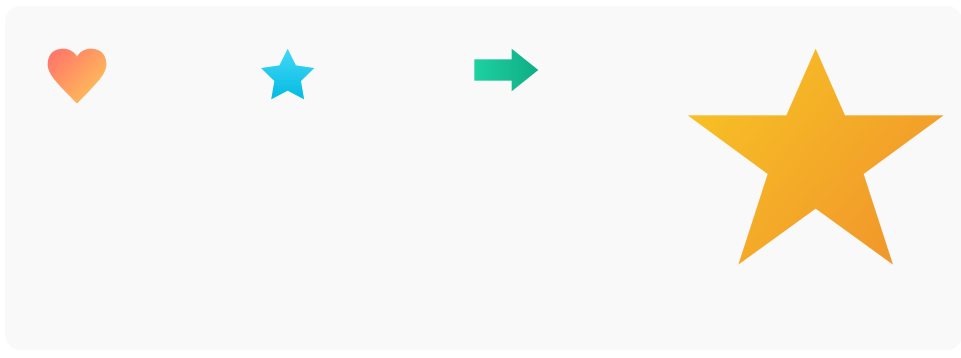
· flipHorizontal / flipVertical

· clipPath: SVG path string

· fill: placeholder color

Path — SVG path with fills

```
Path("M 0 10 L 20 0 L 40 10 L 20 20 Z")
  .fill("linear-gradient(135deg, #ff6b6b, #feca57)")
  .stroke("#333")
  .strokeWidth(2)
  .size(80, 80)
```



Path props:

· d: SVG path string

· fill: color or gradient

· stroke: color

· strokeWidth: number

· size(w, h?) — fit bounds

ClipGroup — clip children to SVG path

```
ClipGroup(
  "M 75 0 L 150 150 L 0 150 Z",   // triangle clip path
  Photo("image.jpg").size(150, 150),
  Text("Clipped!").color("white").padding(8),
).size(150, 150).overflow("hidden")
```



ClipGroup clips all children to an SVG path shape.

· Triangle, star, diamond...



· Any valid SVG path string
· Mix Photo + Text inside
· overflow('hidden') for tight clipping

08 — TABLES

Table, TableRow & TableCell

Structured data rendering with per-cell styling, column spans, and TextDefault inheritance.

Basic table

```
Table(
  TableRow(
    TableCell(Text("Name")),
    TableCell(Text("Role")),
  ).bg("#f0f0f0"),
  TableRow(
    TableCell(Text("Alice")),
    TableCell(Text("Engineer")),
  ),
).borderWidth(1).borderColor("#ccc").spacing(24, 10)
```

Name	Role	Status
Alice Chen	Engineer	Active
Bob Smith	Designer	Active
Carol Wu	Manager	Away

Table structure:
· Table(...rows)
· TableRow(...cells)
· TableCell(...nodes)

.spacing(h, v) sets cell padding.

.borderWidth / .borderColor for outer border.

Column & row spanning

```
TableCell(Text("Spans 2 cols")).colSpan(2)
TableCell(Text("Spans 2 rows")).rowSpan(2)
```

Header spans 2 cols		Col C
A1	Spans 2 rows	C1
A2		C2

.colSpan(n) — merge cols
.rowSpan(n) — merge rows

TableRow also supports:
· .bg() — row background
· .color() via TextDefault
· .weight() via TextDefault

TableCell supports:

· .alignItems() — vertical

· .bg() — cell background

09 — LISTS

List & ListItem

Ordered and unordered lists with built-in marker styles or fully custom Span markers.

Built-in styles

```
List(...items).listStyle("disc")
List(...items).listStyle("circle")
List(...items).listStyle("decimal")
```

disc

- First item
- Second item
- Third item

circle

- First item
- Second item
- Third item

decimal

1. First item
2. Second item
3. Third item

Custom Span markers

```
List(...items)
  .listStyle(Span("→").color("blue"))

// Dynamic per-item:
.listStyle((i) =>
  Span(`${i+1}.`).color(colors[i]))
```

Span marker

→ Install the package

→ Configure settings

→ Render your layout

Function marker

1. High priority task
2. Medium priority task
3. Low priority task

Nested lists

```
List(
  ListItem(
    Text("Parent item"),
    List(
      ListItem(Text("Child A")),
      ListItem(Text("Child B")),
    ).listStyle("circle").markerGap(6).gap(2).marginTop(4),
  ),
).listStyle("disc").markerGap(8).gap(8)
```

→ Page Layout

- pageHeight — enable multi-page
- header / footer — repeating bands
- PageBreak() — explicit breaks

→ Typography

List props:

· listStyle(style | Span | fn)

· startIndex(n) — first index for numbered

· markerGap(px) — space between marker & text

· markerOffset(px) — vertical alignment

- tabStops() — column alignment
- orientation(90|270) — rotated text

· gap(px) — spacing between items

ListItem: any SoneNode children

including nested List nodes.

10 — MULTI-PAGE PDF

Pagination, Headers & Footers

When pageHeight is set, Sone automatically splits content across pages with optional repeating header/footer bands.

Enabling multi-page output

```
await sone(root, {
  pageHeight: 1056,        // A4/Letter page height in px
  width: 816,              // optional canvas width
  margin: { top: 16, bottom: 16 }, // space around header/footer
}).pdf();
```

Header & footer

```
// Static node — same on every page
const header = Row(
  Text("My Document").size(10),
  Text("Confidential").size(9).color("gray"),
).justifyContent("space-between").padding(8, 16)
  .borderWidth(0,0,1,0).borderColor("#e5e5e5");

// Dynamic — receives { pageNumber, totalPages }
const footer = ({ pageNumber, totalPages }: SonePageInfo) =>
  Row(
    Text("My Corp").size(9),
    Text(`${pageNumber} / ${totalPages}`).size(9),
  ).justifyContent("space-between").padding(8, 16)
    .borderWidth(1,0,0,0).borderColor("#e5e5e5");

await sone(root, { pageHeight, header, footer }).pdf();
```

pageBreak prop

Value	Effect
"before"	Force a page break before this element
"after"	Force a page break after this element
"avoid"	Keep element on one page — never split

```
// Explicit break node
PageBreak()  // alias for Column().height(0).pageBreak("before")

// Prop on any node
Column(...).pageBreak("avoid")
Column(...).pageBreak("before")
```

lastPageHeight

"uniform"	"content"
All pages have the same height. Default — required for multi-page PDF.	Last page canvas is trimmed to actual content height. Ideal for individual page images.

11 — QR CODES & ADVANCED

QR Codes, Tab Stops, Text Orientation & Clip Image

Built-in QR code generation and advanced text layout features.

qrcode() — built-in QR generation

```
import { qrcode, Photo } from 'sone';

// Returns SVG as Buffer
const svg = qrcode("https://example.com", {
  pixelSize: 6,
  blackColor: "#000000",
  whiteColor: "#ffffff",
});

// Embed in layout as a Photo node
Photo(svg).size(120).scaleType("cover")
```

QR Code (SVG)



qrcode(data, options?) returns a Buffer containing an SVG QR code ready to embed as a Photo node.

Options:

· pixelSize — module size in px (default 4)

· blackColor — dark module color

· whiteColor — light module color

Tab stops — column alignment with \t

```
Text("Name\tScore\tRank")
  .tabStops(180, 280)      // x positions
```

```
.tabLeader(".")       // dot leader fill
```

Name	Score	Rank
Alice Chen	9,840	1st
Bob Smith	8,120	2nd
Carol Wu	7,655	3rd

Text orientation — layout-aware rotation

```
Text("Rotated").size(16).orientation(90)  // swaps width/height in Yoga
```

0°
0°

90°
90°

180°
180°

270°
270°

clipImage — image through letterforms

```
Text("SONE")
  .size(90)
  .weight("bold")
  .clipImage(Photo("image.jpg").scaleType("cover"))
```

clipImage() fills text letterforms with an image.

The Photo is clipped to the exact shape of each glyph.

Any scaleType works: cover / contain / fill.

12 — API REFERENCE

Complete Node & Builder Reference

All exported node factory functions and their key builder methods.

Node factory functions

Column(...children)
Vertical flex container

Row(...children)
Horizontal flex container

Grid(...children)
CSS Grid container

Text(...parts)
Text block with inline spans

Span(text)
Inline styled text segment

TextDefault(...children)
Set default text styles for subtree

Photo(src) Image node	Path(d) SVG path node
Table(...rows) Table container	TableRow(...cells) Table row
TableCell(...children) Table cell	List(...items) List container
ListItem(...children) List item	ClipGroup(path, ...children) SVG clip group
PageBreak() Explicit page break	qrcode(data, opts?) QR code → Buffer (SVG)
Font.load(name, src) Register custom font	sone(root, config?) Create render context

LayoutProps — all layout nodes

position()	static | relative | absolute	inset / top / left / right / bottom	Absolute position offsets
display()	none | flex | contents	overflow()	visible | hidden | scroll
bg() / background()	Colors, gradients, images	opacity(0–1)	Element opacity
shadow(...css)	CSS box shadows	blur / brightness / contrast...	CSS filters
rotate(deg)	Rotation in degrees	scale(x, y?)	Scale transform
translateX / translateY	Translation	tag(name)	Debug identifier
pageBreak()	before | after | avoid	aspectRatio(n)	Width/height ratio
borderWidth(...)	CSS shorthand 1–4 values	borderColor(value)	Border color
rounded(...) / cornerRadius(...)	Corner radii	cornerSmoothing(0–1)	Squircle smoothing
corner('cut')	Chamfered corners		

Sone — Declarative Canvas Layout for Node.js & the browser
github.com/yourusername/sone · MIT License